

FDMJ Final Report

22300240022 王镜凯

FDMJ Final Report

1. 引言 (Introduction)

整体FDMJ编译流程

1. 词法/语法分析 (源代码 → AST)
2. AST → XML
3. 语义分析
4. IR 规范化 (Canonicalization)
5. IR → Quad
6. 基本块划分 (Blocking)
7. Quad → SSA 形式
8. 寄存器分配准备
9. 寄存器分配 (图着色)
10. Quad → RPI 汇编

流程分析

2. 词法与语法分析 (Lexing & Parsing)

2.1 词法分析器 (lexer.ll)

主要结构

2.2 语法分析器 (parser.yy)

主要结构

3. 类型检查 (Type Checking)

3.1 类型检查机制

类型检查流程与算法

如何处理作用域与继承关系

3.2 类型表构建

符号表与类表结构说明

3.3 错误报告

类型错误的定位与报告方式

示例中的类型检查示例与错误处理

4. AST 到中间表示 IR+ 的转换 (Translation to IR+)

4.1 IR+ 简介与设计选择

IR+ 的结构定义

与 AST 的关系

4.2 转换过程

各类 AST 节点如何转换为 IR+

主要节点转换举例

4.3 辅助表结构

方法表、变量表等 IR 生成所用的数据结构

5. 指令选择与 Quad 表示 (Instruction Selection for Quad)

Tile 设计原则与实现

使用的 tile 模板举例

“def/use” 集的确定

每种 Quad 指令的 def/use 集计算方式

6. 静态单赋值形式 (Static Single Assignment - SSA)

6.1 SSA 转换步骤

1. 删除不可达基本块

2. 插入 ϕ 函数 (placePhi)

3. 变量重命名 (renameVariables)	
6.2 ϕ 函数插入位置的确定	
基于支配树的 ϕ 函数插入策略	
数据流分析应用说明	
7. Quad 优化与 RPi 翻译准备：活跃性分析 (Liveness Analysis)	
7.1 控制流图 (CFG)	
7.2 活跃性分析	
7.3 干涉图构建 (Interference Graph)	
干涉图构造过程与应用	
8. 寄存器分配 (Register Allocation)	
8.1 基于图着色的寄存器分配	
1. 简化 (Simplify)	
2. 合并 (Coalesce)	
3. 冻结 (Freeze)	
4. 溢出 (Spill)	
5. 选择 (Select)	
8.2 寄存器使用策略	
调用者/被调用者保存寄存器处理	
r0-r8 寄存器使用、r9-r10 溢出处理说明	
9. RPi (ARM) 汇编生成 (RPi Assembly Generation)	
9.1 汇编代码生成流程	
Quad 到 ARM 指令的映射策略	
指令选择与优化策略	
9.2 栈帧结构与管理	
方法调用与返回的栈帧布局	
参数传递、局部变量管理、溢出变量保存	
10. 总结与使用说明 (Conclusion & User Manual)	
10.1 本学期工作总结	
10.2 编译器构建与使用指南	

1. 引言 (Introduction)

整体FDMJ编译流程

1. 词法/语法分析 (源代码 → AST)

- 输入：.fmj 源代码文件
- 过程：调用 fdmjParser 进行词法和语法分析，生成抽象语法树 (AST)。
- 输出：AST (内存中的树结构)

2. AST → XML

- 过程：将 AST 转换为 XML 格式，便于后续处理和调试。
- 输出：*.2.ast 文件 (AST 的 XML 表达)

3. 语义分析

- 过程：
 - 读取 AST XML，重新生成 AST。
 - 构建名字映射（符号表），进行类型检查、作用域检查等语义分析。
 - 生成带有语义信息的 AST。
- 输出：*.2-semant.ast 文件（带语义信息的 AST XML）

4. IR 规范化（Canonicalization）

- 过程：对 IR 进行规范化处理（如消除副作用、标准化表达式等）。
- 输出：*.3-canon.irp 文件（规范化后的 IR）

5. IR → Quad

- 过程：将规范化后的 IR 转换为四元式（Quad）中间代码。
- 输出：QuadProgram 对象（内存中）

6. 基本块划分（Blocking）

- 过程：将四元式划分为基本块，便于后续的数据流分析和优化。

7. Quad → SSA 形式

- 过程：将四元式转换为 SSA（静态单赋值）形式，便于寄存器分配和优化。
- 输出：*.4-ssa.quad 文件（SSA 形式的四元式）

8. 寄存器分配准备

- 过程：对 SSA 形式的四元式做一些准备工作，为寄存器分配做铺垫。

9. 寄存器分配（图着色）

- 过程：对四元式进行寄存器分配（通常用图着色算法），并生成寄存器分配结果的 XML。
- 输出：*.4-xml.clr 文件（寄存器分配结果）

10. Quad → RPI 汇编

- 过程：根据寄存器分配结果，将四元式转换为目标平台（如 RPI）的汇编代码。
- 输出：*.s 文件（汇编代码）

流程分析

- 将以以下代码作为示例，分析FDMJ编译流程

```
public int main() {  
    int x;  
    x = 1;  
    return(x);  
}
```

2. 词法与语法分析 (Lexing & Parsing)

2.1 词法分析器 (lexer.ll)

主要结构

- 正则表达式规则区：定义了各种 token 的正则表达式和对应的返回值。

2.2 语法分析器 (parser.yy)

主要结构

- token 和类型声明：定义所有终结符、非终结符及其类型。
- 语法规则区：描述语言的语法结构和归约动作。

经过Lexing & Parsing之后变为以下形式

```
<?xml version="1.0" encoding="UTF-8"?>  
<Program>  
  <MainMethod>  
    <VarDeclList>  
      <VarDecl>  
        <Type typeKind="INT" />  
        <IdExp id="x" />  
      </VarDecl>  
    </VarDeclList>  
    <StmList>  
      <Assign>  
        <IdExp id="x" />  
        <IntExp val="1" />  
      </Assign>  
      <Return>  
        <IdExp id="x" />  
      </Return>  
    </StmList>  
  </MainMethod>  
  <ClassDeclList />  
</Program>
```

3. 类型检查 (Type Checking)

3.1 类型检查机制

类型检查流程与算法

- 首先调用 `makeNameMaps(node)` 构建名字映射（符号表/类型表）。
- 然后创建 `AST_Semant_Visitor` 访问整个 AST，递归遍历每个节点，进行类型检查和语义分析。
- 遍历方式：采用访问者模式（Visitor Pattern），每种 AST 节点（如 `Assign`, `If`, `While`, `CallStm`, `Return` 等）都有对应的 `visit` 方法。
- 类型兼容性检查：核心函数为 `check_compatible_types`，支持基本类型、类类型、数组类型的兼容性判断，支持继承关系的 `upcast`。

如何处理作用域与继承关系

- 作用域：通过 `current_class`、`current_method` 等全局变量记录当前分析的类和方法，辅助查找变量/方法的定义。
- 继承关系：
 - `name_maps->get_ancestors(class_name)` 获取某个类的所有祖先类。
 - 类型检查时，允许子类对象赋值给父类类型（`upcast`），方法查找也会递归父类链

3.2 类型表构建

符号表与类表结构说明

- `Name_Maps`：核心符号表/类型表结构，记录所有类、方法、变量的信息。
- 通过 `makeNameMaps(node)` 构建，遍历 AST，收集所有类、方法、变量声明。
- 提供接口如：
 - `is_method(class, method)`：判断某类是否有某方法
 - `get_method_formal_list(class, method)`：获取方法参数和返回类型
 - `get_class_var(class, var)`：获取类成员变量
 - `get_method_var(class, method, var)`：获取方法局部变量
 - `get_method_formal(class, method, var)`：获取方法参数
 - `get_all_classes()`：获取所有类名
 - `get_ancestors(class)`：获取类的继承链

3.3 错误报告

类型错误的定位与报告方式

- 定位：每个 AST 节点都带有位置信息（`getPos()->print()`），错误报告时会输出出错的源代码位置。

- 报告方式：遇到类型错误时，直接输出错误信息到 cerr，并调用 exit(EXIT_FAILURE) 终止编译。

示例中的类型检查示例与错误处理

- 赋值类型检查 (visit(Assign* node)) :
 - 检查左值是否为 lvalue。
 - 检查左右类型是否兼容（支持类的 upcast）。
- 错误示例

```
if (!left_sem->is_lvalue()) {
    cerr << "Error at " << node->getPos()->print() << ": Left side of assignment must
be an lvalue" << endl;
    exit(EXIT_FAILURE);
}
if (!check_compatible_types(...)) {
    cerr << "Error at " << node->getPos()->print() << ": Type mismatch in assignment"
<< endl;
    exit(EXIT_FAILURE);
}
```

```
//经过semant analyzer后的代码
<?xml version="1.0" encoding="UTF-8"?>
<Program>
  <MainMethod>
    <VarDeclList>
      <VarDecl>
        <Type typeKind="INT"/>
        <IdExp id="x"/>
      </VarDecl>
    </VarDeclList>
    <StmList>
      <Assign>
        <IdExp s_kind="Value" typeKind="INT" lvalue="true" id="x"/>
        <IntExp s_kind="Value" typeKind="INT" lvalue="false" val="1"/>
      </Assign>
      <Return s_kind="Value" typeKind="INT" lvalue="false">
        <IdExp s_kind="Value" typeKind="INT" lvalue="true" id="x"/>
      </Return>
    </StmList>
  </MainMethod>
</ClassDeclList/>
</Program>
```

4. AST 到中间表示 IR+ 的转换 (Translation to IR+)

4.1 IR+ 简介与设计选择

IR+ 的结构定义

- IR+ (Intermediate Representation Plus) 是你编译器的中间代码，采用树状结构 (tree IR)，类似于经典的“树形中间代码”。
- 主要结构体包括：
- tree::Program：整个程序的 IR 表示，包含所有函数声明。
- tree::FuncDecl：函数声明，包含函数名、参数、基本块、返回类型等。
- tree::Block：基本块，包含标签和语句列表。
- tree::Stm/tree::Exp 及其子类：各种语句和表达式（如 Move, Binop, Call, Mem, Const, LabelStm, Jump, Cjump 等）。

与 AST 的关系

- AST (抽象语法树) 表达的是源程序的语法结构，保留了所有语法细节。
- IR+ 是对 AST 的“消解”与“规范化”，更接近机器/汇编层面，去除了语法糖，便于后续优化和代码生成。

4.2 转换过程

各类 AST 节点如何转换为 IR+

- 采用访问者模式，ASTToTreeVisitor 递归遍历 AST，每种节点类型有对应的 visit 方法，生成 IR+ 结构。

主要节点转换举例

- Program/MainMethod/ClassDecl/MethodDecl
 - 生成 tree::FuncDecl，每个方法/主函数都变成一个 IR+ 函数。
 - 方法参数、局部变量、this 指针等都分配唯一的临时变量 (Temp)。
- 变量声明 (VarDecl)
 - 普通变量：分配临时变量，若有初值，生成 Move 语句初始化。
 - 数组变量：分配内存 (malloc)，存储长度，逐元素初始化。
 - 类变量：分配对象内存，递归初始化父类成员和方法表。
- 表达式与语句
 - 赋值 (Assign)：生成 Move 语句。
 - 条件/循环 (If/While)：生成标签、条件跳转 (Cjump)、跳转 (Jump)、块结构。
 - 方法调用 (CallStm/CallExp)：生成 Call，参数列表包括 this 指针和实际参数，虚方法表通过偏移访问。
 - 数组访问 (ArrayExp)：生成下标检查、偏移计算、内存访问。

4.3 辅助表结构

方法表、变量表等 IR 生成所用的数据结构

- Class_table
 - 记录所有类的成员变量和方法在对象内存中的偏移 (var_pos_map, method_pos_map)。
 - 用于对象成员/方法的地址计算。
- Method_var_table
 - 记录每个方法的局部变量、参数与分配的临时变量 (var_temp_map)，以及类型 (var_type_map)。
 - 支持 this 指针、参数、局部变量的统一管理。
- Temp_map
 - 生成唯一的临时变量 (Temp) 和标签 (Label)，保证 IR+ 中变量唯一性。
- Name_Maps
 - 由语义分析阶段生成，提供类、方法、变量的查找、继承关系等信息，辅助 IR 生成。

```
//转换为IR+
<?xml version="1.0" encoding="UTF-8"?>
<Program>
  <FunctionDeclaration name="_^main^_^main" return_type="INT" last_temp="101"
last_label="100">
    <Blocks>
      <Block entry_label="100">
        <Sequence>
          <Label label="100"/>
          <Move>
            <Temp type="INT" temp="100"/>
            <Const value="1"/>
          </Move>
          <Return>
            <Temp type="INT" temp="100"/>
          </Return>
        </Sequence>
      </Block>
    </Blocks>
  </FunctionDeclaration>
</Program>
```

5. 指令选择与 Quad 表示 (Instruction Selection for Quad)

Tile 设计原则与实现

- Tile (模板) 设计原则：将 IR+ (树形中间代码) 的每种常见结构 (如 Move、Binop、Mem、Call、Jump、CJump 等) 映射为一种 Quad 指令。每种 Quad 指令对应一种“树模式”。
- 实现方式：采用访问者模式 (Tree2Quad::visit)，每种 IR+ 节点类型有对应的 visit 方法，进行模式匹配和转换。
- 目标：将复杂的树结构“平铺”为线性、易于后续优化和分配寄存器的四元式序列。

使用的 tile 模板举例

- Move（赋值）：
 - `temp <- temp` → QuadMove
 - `temp <- mem(temp)` → QuadLoad
 - `mem(temp) <- temp` → QuadStore
 - `temp <- temp op temp` → QuadMoveBinop
 - `temp <- call` → QuadMoveCall
 - `temp <- extcall` → QuadMoveExtCall
- 表达式：
 - `temp <- temp op temp` → QuadMoveBinop
 - `temp <- const` → QuadMove
 - `temp <- mem(temp)` → QuadLoad
- 控制流：
 - Label → QuadLabel
 - Jump → QuadJump
 - CJump → QuadCJump
 - Return → QuadReturn
- 函数/外部调用：
 - `call` → QuadCall（无返回值）
 - `extcall` → QuadExtCall（无返回值）
 - `temp <- call` → QuadMoveCall
 - `temp <- extcall` → QuadMoveExtCall

“def/use” 集的确定

每种 Quad 指令的 def/use 集计算方式

- def 集：本条指令定义（写入）的临时变量集合。
- use 集：本条指令使用（读取）的临时变量集合。
- 这些集合用于后续的数据流分析、活跃变量分析和寄存器分配。

```
Function _^main^_main() last_label=100 last_temp=101:
Block: Entry Label: L100
Exit labels:
LABEL L100; def: use:
MOVE t10000:INT <- Const:1; def: 10000 use:
MOVE t0:INT <- t10000:INT; def: 0 use: 10000
RETURN t0:INT; def: use: 0
```

6. 静态单赋值形式 (Static Single Assignment - SSA)

6.1 SSA 转换步骤

1. 删除不可达基本块

- deleteUnreachableBlocks: 移除 CFG（控制流图）中不可达的基本块，保证后续分析只处理可达代码。

2. 插入 ϕ 函数 (placePhi)

- 对于每个变量，找到其所有定义所在的基本块集合 defBlocks。
- 对每个定义块，遍历其支配前沿 (dominance frontier) 上的块，如果该变量在该块的 liveout 集合中且还未插入 ϕ ，则在该块插入 ϕ 函数。
- ϕ 函数的参数为该变量在所有前驱块的值。
- 采用迭代法，直到所有需要插入 ϕ 的位置都被处理。

3. 变量重命名 (renameVariables)

- 为每个变量维护一个计数器和栈，遍历 CFG 的支配树 (dominator tree)。
- 每遇到一次定义 (def)，分配新版本号，压栈。
- 每遇到一次使用 (use)，用栈顶的最新版本号替换。
- ϕ 函数的参数也会被重命名为前驱块中该变量的最新版本。
- 递归处理支配树的子节点，回溯时弹出栈顶，恢复现场。

6.2 ϕ 函数插入位置的确定

基于支配树的 ϕ 函数插入策略

- 支配树 (Dominator Tree)：每个基本块的父节点是唯一支配它的最近块。
- 支配前沿 (Dominance Frontier)：对于每个块 B，B 的支配前沿是所有满足“B 支配其前驱但不支配自身”的块集合。
- 插入策略：
 - 对于每个变量 v，找到所有定义 v 的块集合。

- 对每个定义块，遍历其支配前沿上的块，如果该块的入口处 v 可能活跃（liveout），则在该块插入 $\phi(v)$ 。
- 迭代直到没有新的 ϕ 需要插入。

数据流分析应用说明

- 活跃变量分析（Liveness Analysis）：用于判断变量在某块入口是否活跃，决定是否需要在该块插入 ϕ 。
- def/use 集：每条指令的 def/use 集合用于数据流分析，辅助 SSA 转换和 ϕ 插入。
- 控制流信息（ControlFlowInfo）：维护 CFG、支配树、支配前沿、前驱/后继等信息，支撑 SSA 转换的所有步骤。

```
Function ^_main^_main last_label=100 last_temp=101:
Block: Entry Label: L100
Exit labels:
LABEL L100; def: use:
MOVE t10000:INT <- Const:1; def: 10000 use:
RETURN t10000:INT; def: use: 10000
```

7. Quad 优化与 RPi 翻译准备：活跃性分析（Liveness Analysis）

7.1 控制流图（CFG）

- 基本块（Basic Block）：一串顺序执行、只有入口和出口的指令序列。只有第一个指令有前驱，最后一个指令有后继（如 jump、cjump、return）。
- 基本块识别：
 1. 遍历 Quad 指令，遇到 label、jump、cjump、return 等分界点，划分新块。
 2. 每个块有唯一入口 label，出口为 jump/cjump/return 或下一个块的 label。
- CFG 构建：
 1. 每个基本块为图的节点。
 2. 若块 A 的结尾是 jump/cjump 到块 B，则 A 指向 B。
 3. 顺序块之间也有边（如没有 jump，直接顺序流向下一个块）。
 4. 记录每个块的前驱（predecessors）和后继（successors）。

7.2 活跃性分析

- 活跃变量分析（Liveness Analysis）：确定每个程序点哪些变量“活跃”（即未来会被用到）。
- 数据流方程（逆向分析）：

- 对每个基本块 B：
 - $in[B] = use[B] \cup (out[B] - def[B])$
 - $out[B] = \cup_{S \in succ[B]} in[S]$
- 其中：
 - $use[B]$: 块 B 中在定义前就被用到的变量集合
 - $def[B]$: 块 B 中被定义的变量集合
 - $succ[B]$: B 的所有后继块
- 迭代算法：
 1. 初始化所有 in/out 集为空。
 2. 反复应用方程，直到所有 in/out 集不再变化（收敛）。

7.3 干涉图构建 (Interference Graph)

干涉图构造过程与应用

- 干涉图 (Interference Graph, IG)：无向图，节点为变量（临时变量），边表示两个变量在某处活跃区间重叠（即不能分配同一个寄存器）。
- 构造过程：
 - 对每条 Quad 指令，分析其 def/use 集。
 - 对于每个 def 的变量 t，和该指令 out 集中的所有变量 s ($s \neq t$)，在 IG 中添加边 t-s。
 - 这样，所有在同一活跃区间的变量都两两相连。

8. 寄存器分配 (Register Allocation)

8.1 基于图着色的寄存器分配

1. 简化 (Simplify)

- 目标：优先移除度数小于 k（可用寄存器数）的节点，压入简化栈。
- 实现：Coloring::simplify() 遍历干涉图，找到度数 $< k$ 且不是机器寄存器/移动相关节点的节点，移除并记录。
- 意义：这些节点在后续分配时总能找到可用颜色（寄存器）。

2. 合并 (Coalesce)

- 目标：合并不冲突的 move 相关节点，减少冗余 move 指令。
- 实现：Coloring::coalesce() 检查 move 对 (u, v)，若合并后不会导致度数过高 (Briggs 策略)，则合并邻居，更新 coalescedMoves。
- 意义：减少 move 指令，提高代码效率。

3. 冻结 (Freeze)

- 目标：无法安全合并的 move 相关节点，移除其 move 关系，转为普通节点，便于后续简化。
- 实现：Coloring::freeze() 找到度数 $< k$ 的 move 相关节点，移除其 move 对。
- 意义：为后续简化创造条件，避免不必要的溢出。

4. 溢出 (Spill)

- 目标：当无法简化/合并/冻结时，选择度数最高的节点标记为溢出 (spill)，即分配到内存。
- 实现：Coloring::spill() 选择度数最大的非机器寄存器节点，移除并记录为 spilled。
- 意义：保证算法能继续进行，牺牲部分变量的性能以保证分配可行。

5. 选择 (Select)

- 目标：为简化栈中的节点分配实际寄存器（颜色），若无可用寄存器则标记为溢出。
- 实现：Coloring::select() 逆序弹出简化栈，为每个节点分配未被邻居占用的颜色。
- 意义：完成实际的寄存器分配。

8.2 寄存器使用策略

调用者/被调用者保存寄存器处理

- 调用者保存 (Caller-saved)：函数调用前，调用者需保存 r0-r3（参数/返回值寄存器）等易被覆盖的寄存器。
- 被调用者保存 (Callee-saved)：函数进入时，被调用者需保存 r4-r8（一般寄存器）等，返回前恢复。
- 实现：在汇编生成阶段，函数入口/出口自动插入 push/pop 指令保存/恢复被调用者保存寄存器。

r0-r8 寄存器使用、r9-r10 溢出处理说明

- r0-r3：参数传递和返回值，调用者保存。
- r4-r8：一般用途寄存器，被调用者保存，分配给大部分临时变量。
- r9-r10：专门用于溢出变量 (spill)，即当变量无法分配到 r0-r8 时，先加载到 r9/r10，再参与运算。
- 代码生成时，遇到 spill 变量，先用 ldr 加载到 r9/r10，操作后再 str 存回内存。

```
<?xml version="1.0" encoding="UTF-8"?>
<COLORING>
  <Coloring func="_^main_^main" k="9">
    <Colors>
      <Color node="0" color="0"/>
      <Color node="1" color="1"/>
      <Color node="2" color="2"/>
      <Color node="3" color="3"/>
      <Color node="10000" color="0"/>
    </Colors>
  </Coloring>
</COLORING>
```

```
<Spills/>
</Coloring>
</COLORING>
```

9. RPi (ARM) 汇编生成 (RPi Assembly Generation)

9.1 汇编代码生成流程

Quad 到 ARM 指令的映射策略

- Quad 指令到 ARM 指令的映射采用了直接模式匹配和模板化生成：
 - QuadMove → mov 或 ldr/str (若涉及溢出变量)
 - QuadMoveBinop → add/sub/mul/sdiv 等
 - QuadLoad → ldr
 - QuadStore → str
 - QuadJump → b
 - QuadCJump → cmp + 条件跳转 (如 beq, bne, blt, bgt 等)
 - QuadCall/QuadMoveCall → 参数准备 (mov), blx 调用, 返回值处理
 - QuadExtCall/QuadMoveExtCall → 参数准备, bl 调用, 返回值处理
 - QuadReturn → 返回值放入 r0, 恢复栈帧, pop 返回
 - QuadPhi → 多路赋值 (SSA 形式下的合流点)
- 溢出变量 (spill) 处理：
 - 若变量被分配到内存 (spilled), 则在用到前先 ldr 到 r9/r10, 用完后 str 回内存。
 - 通过 color->spills 判断变量是否溢出, 决定是否插入 ldr/str。
- 标签与跳转：
 - 所有标签、跳转目标都加上函数名前缀, 避免全局冲突。

指令选择与优化策略

- 冗余指令消除：
 - 检查 mov rX, rX 等冗余赋值, 直接跳过。
- 检查连续的 add + load/store, 合并为带偏移的 ldr/str 指令 (如 ldr rX, [rY, #imm]), 减少指令数。
- 参数传递优化：
 - 若参数已在目标寄存器, 无需重复 mov。
- 跳转优化：

- 若跳转后紧跟目标标签，去除多余的跳转指令

9.2 栈帧结构与管理

方法调用与返回的栈帧布局

- 函数入口：
 - `push {r4-r10, fp, lr}`: 保存被调用者保存寄存器、帧指针、返回地址。
 - `add fp, sp, #32`: 设置帧指针 (fp)，为局部变量和溢出变量预留空间。
 - 若有溢出变量 (spill)，`sub sp, sp, #spill_size`: 为溢出变量分配栈空间。
- 函数出口：
 - `sub sp, fp, #32`: 恢复栈指针。
 - `pop {r4-r10, fp, pc}`: 恢复寄存器、帧指针，返回。

参数传递、局部变量管理、溢出变量保存

- 参数传递：
 - 前 4 个参数通过 r0-r3 传递，超出部分通过栈传递（本实现只处理前 4 个）。
 - 调用前将参数 mov 到对应寄存器。
- 局部变量管理：
 - 优先分配到 r4-r8 等寄存器。
 - 若寄存器不足，变量溢出到栈，由 `color->spills` 记录。
- 溢出变量保存：
 - 每个溢出变量分配 4 字节栈空间，访问时用 `ldr/str` 加载/保存到 r9/r10。
 - 通过 `color->get_spill_offset(temp_num)` 计算变量在栈帧中的偏移。

```
.section .note.GNU-stack

@ Here is the RPI code

@ Here's function: __main__main

.balign 4
.global main
.section .text

main:
    push {r4-r10, fp, lr}
    add fp, sp, #32
main$L100:
    mov r0, #1
    sub sp, fp, #32
    pop {r4-r10, fp, pc}
```

```
.global malloc
.global getint
.global putint
.global putch
.global putarray
.global getch
.global getarray
.global startttime
.global stoptime
```

10. 总结与使用说明 (Conclusion & User Manual)

10.1 本学期工作总结

1. 前端：词法、语法与语义分析

- 词法分析：用 Flex将源代码分割为 Token。
- 语法分析：用 Bison将 Token 组织为 AST（抽象语法树），支持类、方法、表达式、控制流等结构。
- 语义分析：构建符号表（Name_Maps），进行类型检查、作用域检查、继承关系处理，保证程序语义正确，发现并报告类型错误。

2. 中间端：IR+ 生成与优化

- AST 到 IR+：采用访问者模式，将 AST 转换为树形中间表示（IR+），规范化控制流、变量、方法调用等。
- 辅助表结构：构建类表、方法表、变量表，辅助 IR 生成和后续优化。
- IR+ 到 Quad：通过 tile（模板）匹配，将树形 IR+ 平铺为线性四元式（Quad），便于后端处理。

3. 优化与分析

- 基本块与控制流图（CFG）：划分基本块，构建 CFG，为数据流分析和优化打下基础。
- 活跃性分析：采用逆向数据流方程，迭代计算每个变量在每条指令的活跃区间。
- 干涉图构建：根据活跃区间构建变量间的干涉图，为寄存器分配做准备。
- SSA 形式：插入 ϕ 函数，变量重命名，极大简化后端优化和寄存器分配。

4. 后端：寄存器分配与汇编生成

- 图着色寄存器分配：实现了简化（Simplify）、合并（Coalesce）、冻结（Freeze）、溢出（Spill）等完整流程，合理分配 r0-r8，溢出变量用 r9/r10 及栈空间。
- 调用约定支持：区分调用者/被调用者保存寄存器，保证跨函数调用的正确性。
- Quad 到 ARM 汇编：模板化生成 ARM 指令，peephole 优化，支持溢出变量的加载/保存，参数传递、返回值处理、栈帧管理等。
- 栈帧结构：严格按照 ARM 规范，支持局部变量、溢出变量、参数、返回地址的高效管理。

10.2 编译器构建与使用指南

- `make build` 构建整个项目, `make clean` 清除build文件夹
- `make compile` 编译test目录下的所有fmj文件
- `make compile-one` 编译test目录下指定的fmj文件 (需修改makefile中的FILE变量)
- `make run` 运行test目录下所有的.s汇编
- `make run-one` 运行test目录下某个指定的汇编
- 输入输出文件格式:

```
bubblesort.fmj
↓
bubblesort.2.ast
↓
bubblesort.2-semant.ast
↓
bubblesort.3.irp
↓
bubblesort.3-canon.irp
↓
bubblesort.4-prepared.quad
↓
bubblesort.4-ssa-xml.quad
↓
bubblesort.4-xml.clr
↓
bubblesort.s
```